Second-Order Systems, and Unity Feedback, PD and PV Controllers

Source and modifications. This manual is an adaptation of two Quanser student workbooks for the QUBE-Servo 2 — *Second-Order Systems* and *PD Control* (QUBE-SERVO 2 Workbook – Student, v1.2, © 2021 Quanser Inc.) — combined into a single laboratory experiment and used under the licence reproduced at the end of this document.

Retained from the original workbooks: from *Second-Order Systems*, the background on the standard second-order step response, on peak time and percent overshoot, and on unity feedback, together with Figures 1 and 2 and Equations (1) to (9), and the unity-feedback procedure in Section 6.2; from *PD Control*, the background on PID control, on derivative kick and on the filtered derivative, together with Figures 3 to 5 and Equations (10) to (16), and the proportional-plus-velocity procedure in Section 6.3, including the gains kp = 2.5 V/rad and kd = 0.05 V·s/rad and the sweeps of kp between 1 and 4 and of kd between 0 and 0.15 V·s/rad.

Modified from the original: the supplied Simulink models have been removed, so you build the models yourself from your earlier work; the default model parameters have been replaced with K = 23.1 rad/s/V and τ = 0.165 s; the procedures have been rewritten so that you build each Simulink model yourself rather than opening a supplied one; and the closing design exercise of the *PD Control* workbook has been expanded into Section 7, which states the design specification explicitly and requires a documented design iteration rather than a single calculation.

Written for ENME 561/661 by Dr. Mahdis Bisheban: the materials list, end-of-session shutdown procedure, objectives and deliverables; Section 6.1 on numerical differentiation of clean and noisy signals; the requirement to design your own Simulink model in each section; and all of Section 7, Controller Design to Specification, including Tables 1 and 2.

Table of Contents

Second-Order Systems, and Unity Feedback, PD and PV Controllers 1

1 Materials 3

2 End-of-Session Shutdown Procedure 3

3 Deliverables 3

4 Objectives 3

5 Background 4

5.1 Second-Order Step Response 4

5.2 Peak Time and Overshoot 5

5.3 Unity Feedback 5

5.4 PID Control 6

5.5 PD and PV Control 7

6 In-Lab Exercises 9

6.1 Numerical Differentiation in Noisy and Rapidly Varying Signals 9

6.2 Second Order System Step Response with the Unity Feedback 10

6.3 Second Order System Step Response with PD and PV Controllers 10

7 Controller Design to Specification 12

7.1 Design requirements and constraints 12

7.2 Predict 12

7.3 Test and revise 13

7.4 Trade-offs 13

7.5 Optional: adding integral action 13

1 Materials

- Hardware: QUBE-Servo 2, Cables, Laptop
- Software: Stall detection block and the qube2_param.m parameter script (available on D2L)
- QUBE-Servo 2 User Manual.pdf (available on D2L)

2 End-of-Session Shutdown Procedure

At the end of each lab session, please ensure the following steps are completed:

1. Make sure the QUARC controller is stopped.
2. Turn off the QUBE-Servo 2.
3. Shut down the laptop.
4. Place the laptop in the designated cabinet.

3 Deliverables

- **Section 6.1, Numerical Differentiation**: submit one **.mlx file** and a **PDF export** of it. Do not use a Simulink model for this section.
- **Section 6.2**: submit your **.slx model file(s)** with the **results and discussion** in the **main PDF**. **Section 6.3**: submit the **.slx model only** — nothing from that section is written up.
- **Section 7, Controller Design to Specification**: submit the completed **Table 1** and **Table 2** inside the **main PDF**.

In total: two PDF files, one .mlx file, and your .slx model file(s).

Answering the questions. Each question is tagged with the form its answer must take — *[number]*, *[figure]*, *[screenshot]*, *[table]* or a stated number of sentences. Steps tagged *[no write-up needed]* are there to be done, not described; the file, figure or screenshot you submit is the evidence. Do not restate the procedure.

Submission checklist. Everything that is graded for Lab 4 is listed here. Tick each row as you complete it.

✓	What to submit	Where	When
	Main report PDF (Sections 6.2 and 7)	D2L Dropbox	Lab 4 deadline
	MATLAB Live Script, .mlx (Section 6.1)	D2L Dropbox	Lab 4 deadline
	PDF export of the Live Script	D2L Dropbox	Lab 4 deadline
	Simulink model(s), .slx (Sections 6.2 and 6.3)	D2L Dropbox	Lab 4 deadline
	Design iteration log, Table 2 (Section 7)	In the main PDF	Lab 4 deadline
	Individual Exit Survey	Form	Lab 4 deadline

4 Objectives

You study the effects of numerical differentiation on clean and noisy signals, and observe how differentiation amplifies noise and highlights sudden changes in a signal. This analysis is important because the derivative term in a PID controller requires taking the derivative of a signal.

You study a second-order system with the unity feedback and PD and PV controllers, and measure the peak time and percent overshoot of the response using both the DC motor model and the physical DC motor.

You design and implement PD and PV controllers with a 1st order filter. You do experimentation to see the effects of different values $\boldsymbol{k_p}$ and $\boldsymbol{k_d}$ on the DC motor behaviour. You relate the effects of $\boldsymbol{k_p}$ and $\boldsymbol{k_d}$ to the natural frequency $\boldsymbol{\omega_n}$ and damping ratio $\boldsymbol{\zeta}$.

From this lab, you learn that the natural frequency ω_n and damping ratio ζ are key characteristics of a system's response. With unity feedback, these parameters are fixed by the system. However, by implementing PD or PV controllers, you can design suitable k_p and k_d values to achieve the desired ω_n and ζ, thereby shaping the system response.

5 Background

5.1 Second-Order Step Response

The *standard second-order* transfer function has the form

$$\frac{Y(s)}{R(s)} = \frac{\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2} \qquad (1)$$

where ω_n is the natural frequency and ζ is the damping ratio. $Y(s) = L[(y(t)]$ is the output and $R(s) = L[r(t)]$ is the input, where L represent the Laplace transform. The properties of the second-order system's response depend on the values of the parameters ω_n and ζ.

Consider a second-order system, as shown in Equation 1, subjected to a step input given by

$$R(s) = \frac{R_0}{s} \qquad (2)$$

with a step amplitude of R_0 = 1.5. The system response to this input is shown in Figure 1, where the red line is the output response $y(t)$ and the blue line is the step input $r(t)$.

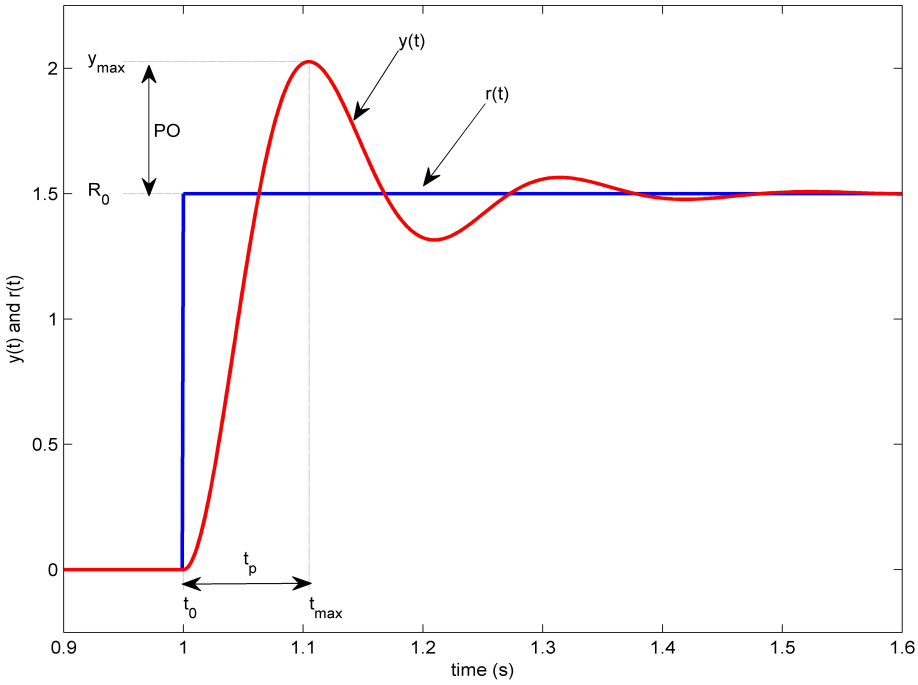


Figure 1. Second-order step response with a step amplitude of $R_0 = 1.5$

5.2 Peak Time and Overshoot

The maximum value of the response is denoted by the variable y_{max} and it occurs at a time t_{max}. For a response similar to Figure 1, the percent overshoot is found using

$$PO = \frac{100(y_{max} - R_0)}{R_0} \quad (3)$$

From the initial step time, t_0, the time it takes for the response to reach its maximum value is

$$t_p = t_{max} - t_0 \quad (4)$$

This is called the *peak time* of the system.

In a second-order system, the amount of overshoot depends solely on the damping ratio parameter (ζ), and it can be calculated using the equation

$$PO = 100e^{\left(-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right)} \quad (5)$$

The peak time depends on both the damping ratio and the natural frequency of the system, and it can be derived as:

$$t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}} \quad (6)$$

5.3 Unity Feedback

The unity-feedback control loop shown in Figure 2 will be used to control the position of the QUBE-Servo 2.

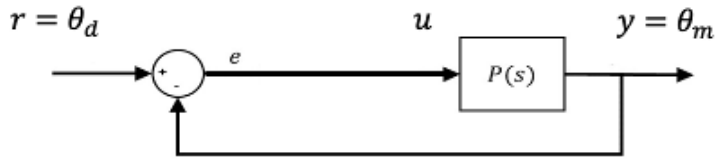


Figure 2 Unity feedback loop

The QUBE-Servo 2 position-to-voltage transfer function is

$$P(s) = \frac{\Theta_m(s)}{V_m(s)} = \frac{K}{s(\tau s+1)} \qquad (7)$$

where K is the model steady-state gain, τ is the model time constant, $\Theta_m(s) = L[\theta_m(t)]$ is the motor / disk position, and $V_m(s) = L[v_m(t)]$ is the applied motor voltage.

The controller is denoted by $C(s)$. If we use unity feedback, therefore $u(t) = e(t)$, which is equivalent to a PID controller set to $k_p = 0, k_d = 0, K_i = 0$, or

$$C(s) = \frac{U(s)}{E(s)} = 1 \qquad (8)$$

The closed-loop transfer function of the QUBE-Servo 2 is denoted $Y(s)/R(s) = \Theta_m(s)/\Theta_d(s)$.

The closed-loop transfer function of the QUBE-Servo 2 position control from the reference input $R(s) = \Theta_d(s)$ to the output $Y(s) = \Theta_m(s)$ using unity feedback, as shown in Figure 2 is

$$\frac{\Theta_m(s)}{\Theta_d(s)} = \frac{\frac{K}{\tau}}{s^2+\frac{1}{\tau}s+\frac{K}{\tau}} \qquad (9)$$

5.4 PID Control

The proportional, integral, and derivative control can be expressed mathematically as follows

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)d\tau + k_d \frac{de(t)}{dt} \qquad (10)$$

The corresponding block diagram is given in Figure 3. The control action is a sum of three terms referred to as proportional (P) (specifically $k_p e(t)$), integral (I) (specifically $k_i \int_0^t e(\tau)d\tau$) and derivative (D) (specifically $k_d \frac{de(t)}{dt}$) control actions. The controller Equation 10 can also be described by the transfer function in the s-domain as follows

$$C(s) = \frac{U(s)}{E(s)} = k_p + \frac{k_i}{s} + k_d s \qquad (11)$$

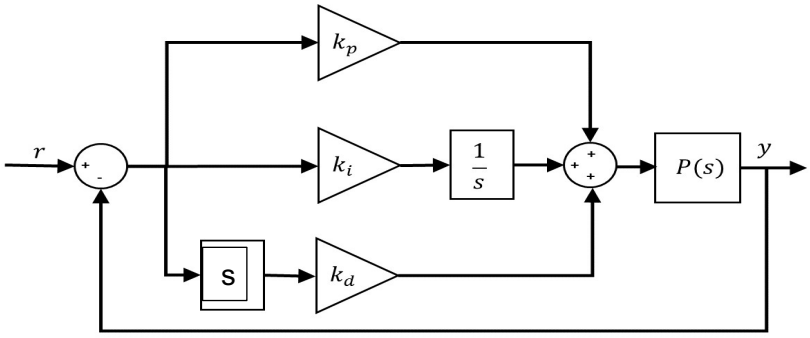


Figure 3 Block diagram of PID control without filter

The functionality of the PID controller can be summarized as follows. The proportional term is based on the present error, the integral term depends on past errors, and the derivative term is a prediction of future errors.

The PID controller described by Equation 10 in the time domain or Equation 11 in the s domain is called a *parallel* PID controller without filtering. However, attempts to implement such a controller may not lead to a good system response for real-world systems. The main reason for this is that measured signals often include measurement noise. Differentiating a noisy measured signal (i.e. in the derivative control) can produce a control signal with high frequency components that yield an undesirable response and, over time, may even damage the actuator (e.g. DC motor). To overcome this issue, we usually use a filter on the derivative action as shown in Figure 4.

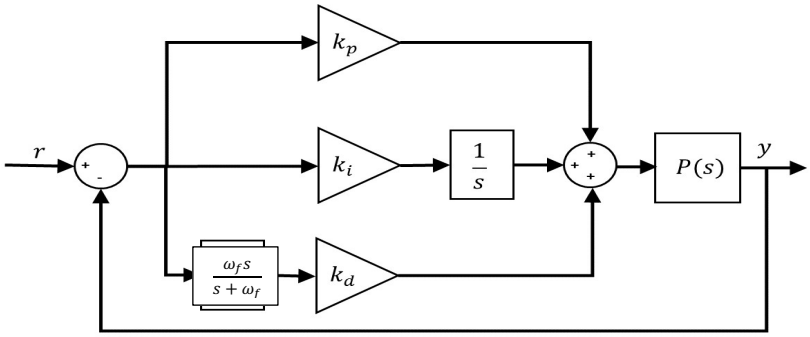


Figure 4. Block diagram of PID control with filter

5.5 PD and PV Control

For this laboratory, the integral term will not be used to control the servo position. A variation of the classic Proportional Derivative (PD) control will be used: the proportional plus rate feedback control illustrated in Figure 5, also known as proportional plus velocity or PV control. Integral action, and the wind-up problem it creates, are covered in the required reading (Franklin, Section 4.3).

Unlike standard PD controllers, only the negative derivative of the output is fed back, rather than the rate of change of the error. This approach avoids a derivative kick, which occurs when there is a sudden change in the reference signal (r), causing a large derivative value (dr/dt). Therefore, instead of using the derivative of the error, $de/dt = d(r - y)/dt$, we use $-dy/dt$.

Further, in the hardware implementation of the control, a low-pass filter will be used in-line with the derivative term to suppress measurement noise.

The combination of a first-order low-pass filter, $\omega_f/(s+\omega_f)$, and the derivative term, s, will be used instead of a direct derivative:

$$D(s) = k_d \frac{\omega_f s}{s+\omega_f} \quad (12)$$

where ω_f is the cut-off frequency of the filter.

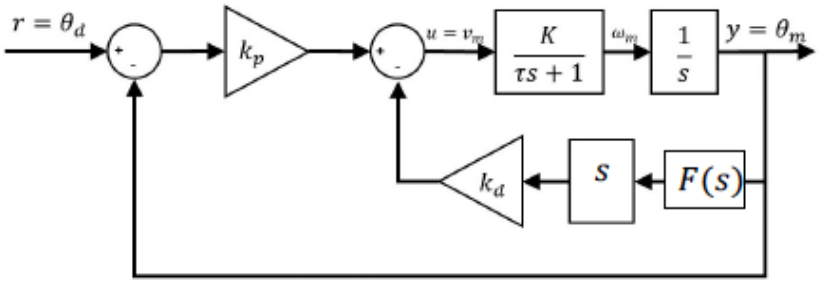


Figure 5 Block diagram of proportional plus rate (PV) feedback control with filter, $F(s) = \frac{\omega_f}{s+\omega_f}$

The proportional plus rate feedback control has the following structure

$$u = k_p\big(r(t) - y(t)\big) - k_d \frac{dy(t)}{dt} \quad (13)$$

where k_p is the proportional gain, k_d is the derivative (velocity) gain, $r = \vartheta_d(t)$ is the setpoint or reference motor/load angle, $y = \vartheta_m(t)$ is the measured load shaft angle, and $u = V_m(t)$ is the control input (applied motor voltage).

Assume all initial conditions are zero, i.e. $\vartheta_m(0^-) = 0$ and $w_m(0^-) = \frac{d\theta m}{dt}(0^-) = 0$, taking the Laplace transform of Equation 13 yields

$$U(s) = k_p\big(R(s) - Y(s)\big) - k_d s Y(s) \quad (14)$$

Assuming that the actuator is not saturated, i.e. $U = V_m$, then, $U(s)$ can be substituted into Equation 7 to result in

$$Y(s) = \frac{K}{s(\tau s+1)}\Big(k_p\big(R(s) - Y(s)\big) - k_d s Y(s)\Big) \quad (15)$$

Solving for $Y(s)/R(s)$, we obtain the closed-loop expression

$$\frac{Y(s)}{R(s)} = \frac{Kk_p}{\tau s^2 + (1+Kk_d)s + Kk_p} \quad (16)$$

This is a second-order transfer function and its form is similar to the standard second-order transfer function shown in equation 1.

6 In-Lab Exercises

6.1 Numerical Differentiation in Noisy and Rapidly Varying Signals

This section is MATLAB only. Write the code in a Live Script (.mlx) and answer the questions in the same file. Do not build a Simulink model and do not wire any QUARC blocks here. The hardware is not used in Section 6.1. A Simulink submission for this section receives zero.

The objective of this section is to study the effects of numerical differentiation on clean and noisy signals, and to observe how differentiation amplifies noise and highlights sudden changes in a signal. Write the code in the mlx-file and provide the answer to this question in the same mlx-file. You cannot use a Simulink file, if you do so, you will receive 0 for this entire section.

You need to first generate two signals, add noise to them, and take the numerical differentiation of them

Specifically, generate two signals:

- S_1 : a square wave signal
- S_2 : a sinusoidal signal

Define an appropriate time vector, e.g $T = (0: 0.001: 1), \Delta t = 0.001$, for both signals.

Add Gaussian noise with a mean of zero and a standard deviation of about 5% of the signal amplitude to both signals.

Denote the noisy signals as:

- S_{n1}: noisy square wave
- S_{n2}: noisy sinusoidal wave

Perform numerical differentiation (e.g., using the *diff* function) on each of the four signals, and denote them as $dS_1/dt, dS_2/dt, dS_{n1}/dt, dS_{n2}/dt$.

Recall, you must take numerical differentiation at time t using the Euler method, given two consecutive data points:

$$dS_1/dt \approx \frac{S_1(t) - S_1(t - \Delta t)}{\Delta t}$$

You cannot use ready to use MATLAB functions. You need to write your numerical differentiation code. *[.mlx file]*

Plot the original signals and their derivatives for comparison. *[figures]*

Report your observations and discuss *[3-5 sentences on each point]*

- Noise Amplification
- Effect of Sudden Changes

6.2 Second Order System Step Response with the Unity Feedback

In this section, you will study the system with unity feedback, i.e. $Y(s)$, or feedback signal gain is 1, and there is no controller on the forward block i.e., ($C(s) = 1$) . You will measure the peak time and percent overshoot of the response using both the DC motor model and the physical DC motor.

1. Given the QUBE-Servo 2 closed-loop equation under unity feedback and the default model parameters given in the data sheet (*K = 23.1 rad/s/V and τ = 0.165 s)*, find the natural frequency and damping ratio of the system. *[derivation + 2 numbers]*
2. Based on the ω_n and ζ obtained, what is the expected peak time and percent overshoot? *[2 numbers]*
3. Design a SIMULINK code that implements the *unity feedback control* in SIMULINK. A step reference (i.e. desired position or setpoint) of 1 rad is applied at 1 second. *[no write-up needed]*
4. Build and run the QUARC controller. *[no write-up needed]*
5. Attach the QUBE-Servo 2 position response - showing both the setpoint and measured positions in one scope - as well as the control signal sent to the motor. *[figure]*
6. Measure the peak time and percent overshoot of the response and compare that with the expected results. You need to show the values of peak overshoot and peak time on the plot. *[2 numbers marked on the figure + 2-3 sentences]*
7. Explain why the *expected* peak time and percent overshoot calculated do not match the measured values exactly. *(For your own understanding — no written answer required.)*
8. Make sure the QUARC controller is stopped. *[no write-up needed]*

6.3 Second Order System Step Response with PD and PV Controllers

For this section, **ensure that the Stall detection block is properly used in the SIMULINK code.**

In this section, you need to add a PD and PV controllers to the Simulink code. For both of these controllers, implement a first-order filter on D or V actions.

Note that you can run one of the controllers at a time.

Nothing in this section is graded. Section 6.3 is bench work: it is where you build the PD and PV controllers, see what kp, kd and the filter cut-off actually do to the motor, and derive the gain expressions you will need next. Do all of it, and keep your plots and notes — but write none of it up. Submit the .slx model, and nothing else from this section. The design, the reasoning and the evidence are assessed in Section 7, which is where the specification is stated and the iteration is recorded.

Before you tune anything on the hardware. Open **PIDGainEffects.mlx** from D2L. It uses a simple $1/(s+1)^3$ plant, so you can change kp, ki and kd and see the effect immediately without waiting for a build — use it to form an expectation before you spend bench time. It also shows how to use the MATLAB **stepinfo** function to extract peak time, percent overshoot and settling time from a response. Use **stepinfo** to report those quantities in this section and in Section 7, rather than reading values off the scope by eye.

1. Begin by implementing PD control to control the servo motor.

Use a derivative transfer function that includes an appropriate cut-off frequency.

What cut-off frequency did you choose? How did you determine this value?

Discuss the relationship between the chosen cut-off frequency and the natural frequency of your system. *(For your own understanding — no written answer required.)*

Note: Selecting the cut-off frequency is equivalent to designing the filter.

Starting point. The Quanser reference implementation of this controller uses a filtered derivative of 100s/(s + 100), that is ωf = 100 rad/s. Start from that value, decide whether it suits your system and the encoder noise you measured in Lab 2, and change it if your reasoning says you should. Section 7.4 asks you to push it an order of magnitude each way.

2. Using the same code, add the **PV** control to control the servo motor. What is the cut-off frequency that you choose? Is it different from the one chosen for the PD controller? *(For your own understanding — no written answer required.)*

3. Set the *Signal Generator block* such that the servo command (i.e. reference angle) is a *square wave* with an amplitude of 0.5 rad and at a frequency of 0.4 Hz. *[no write-up needed]*

4. Set the proportional gain to k_p = 2.5 V/rad and the derivative gain to *kd* = 0.05 V·s/rad. *[no write-up needed]*

5. Build and run the QUARC controller for both the PD and PV controllers. Show a graph of the two outputs, one for each controller, with the reference and control signals. *(For your own understanding — no written answer required.)*

6. Discuss your observation and compare the PV and PD controllers. *(For your own understanding — no written answer required.)*

7. With only the PV controller, Set k_p = 2.5 V/rad. Keep the derivative gain at 0 and vary k_p between 1 and 4. Show the plots. How does the proportional gain affect the servo position control response? *(For your own understanding — no written answer required.)*

8. With only the PV controller, Set k_p = 2.5 V/rad and vary the derivative gain k_d between 0 and 0.15 V/(rad/s). Show the plots. How do different values of derivative gain affect the servo position control response? *(For your own understanding — no written answer required.)*

9. Find the proportional and derivative gains required for the QUBE-Servo 2 closed-loop transfer function given in Equation 16 to match the standard second-order system in Equation 1. In other words, present k_p and k_d as a function of ω_n and ζ. Please show the derivation. *(For your own understanding — no written answer required.)*

10. Based on your observation of varying k_p and k_d , design k_p and k_d. Discuss your rationale for the designed values. *(For your own understanding — no written answer required.)*

With the design, what are the values of ωn and ζ? How are these values different from the ones you calculated in the other section? *(For your own understanding — no written answer required.)*

11. Implement these ωn and ζ in the Simulink code without running the hardware, and simulate the response of such a system. Plot the system response. Do the values of ωn and ζ match your observations from the graphs? Discuss these. *(For your own understanding — no written answer required.)*

12. Stop the QUARC controller. *[no write-up needed]*

7 Controller Design to Specification

In Section 6.3 you observed how kp and kd change the response. In this section you design them to meet a stated specification, and you keep revising until the hardware meets it or you can explain why it cannot. Work with the PV controller and the first-order filter.

7.1 Design requirements and constraints

Before you build anything, complete Table 1. An experiment design begins with what you are required to achieve and what you are allowed to do, not with the steps themselves. You will not be able to answer every row from this manual alone: the QUBE-Servo 2 User Manual on D2L and your own Lab 3 results contain what you need.

Table 1: Design requirements and constraints [filled out Table]

Item	Your entry	Source or justification
Percent overshoot, PO	≤ 2.5 %	Given
Peak time, tp	≤ 0.15 s	Given
Peak motor voltage, |Vm|	must not saturate	Given
Amplifier saturation limit (V)		
Reference signal: amplitude and frequency		
Encoder resolution, and the smallest angle you can resolve		
Filter cut-off frequency, ωf		
K and τ computed from your Lab 3 parameters		
ωn and ζ implied by the specification above		
kp and kd implied by that ωn and ζ		

7.2 Predict

1. Convert the specification on percent overshoot and peak time into a target natural frequency and damping ratio, then into kp and kd. Reproduce the two gain expressions from Section 6.3 here, since that section is not submitted, and show the calculation. *[derivation + 4 numbers]*

2. Recompute K and τ from the motor resistance and back-emf constant you identified in Lab 3, and repeat the calculation. How different are the two sets of gains, and which set would you trust more? *[numbers + 3-5 sentences]*

7.3 Test and revise

3. Implement your first design on the hardware. Record the measured percent overshoot, the measured peak time and the peak control voltage. *[no write-up needed]*

4. Revise. You will need at least three iterations, and your first design will almost certainly not meet the specification. Record every iteration in Table 2, including the ones that failed. The last column is what is graded: a one-line reason for each change, written before you run it, not after. *[table, at least 3 rows]*

Table 2: Design iteration log

#	kp	kd	ωf	Predicted PO / tp	Measured PO / tp	Peak Vm	What I changed, and why
1							
2							
3							

5. State whether your final design meets every row of Table 1. If it does not, name the requirement you could not meet and explain why. A design that misses the specification and diagnoses the reason correctly is worth more here than one that meets it without explanation. *[3-5 sentences]*

7.4 Trade-offs

6. Using your own iteration log as evidence, explain the three trade-offs you had to balance: speed against overshoot, damping against noise amplification, and filter cut-off frequency against phase lag. Refer to specific iterations by number. *[4-6 sentences]*

7. Set your filter cut-off frequency one order of magnitude above, and then one order of magnitude below, the value you chose. Run both and show the responses. Connect what you see to the encoder noise you measured in Lab 2. *[2 figures + 3-5 sentences]*

8. Stop the QUARC controller. *[no write-up needed]*

7.5 Optional: adding integral action

This subsection is optional and is not graded. Attempt it only after your design in Section 7.3 meets the specification. Keep the kp and kd you settled on and change nothing else, so that anything you see here is caused by the integral term and not by your proportional or derivative gains.

9. Add an integral term to your controller and raise ki from zero in small steps, keeping kp and kd fixed. Watch what happens to the percent overshoot, the peak time and the settling time. *(For your own understanding — no written answer required.)*

10. In theory your plant already contains an integrator, so the steady-state error to a step reference should be zero without any integral term. In practice it is not: Coulomb friction means the motor cannot move until the control voltage exceeds the breakaway voltage you measured in Lab 3, so the shaft can come to rest a small distance short of the setpoint and stay there. Measure that residual error with ki = 0, then raise ki and measure it again. Repeat both while pressing gently on the inertia disc to apply a constant disturbance torque. This is where integral action earns its place on a plant that is already type 1. *(For your own understanding — no written answer required.)*

11. Keep raising ki until the response becomes sluggish and the overshoot grows rather than shrinks. The control signal is saturating at the amplifier limit while the integrator keeps accumulating error, so the integral term has to unwind before the motor can reverse. This is integral wind-up. Note the value of ki at which you first see it, and watch the control-signal scope rather than the position scope to catch it. *(For your own understanding — no written answer required.)*

12. Stop the QUARC controller. *[no write-up needed]*

© 2021 Quanser Inc., All rights reserved.

Quanser Inc.

119 Spy Court

Markham, Ontario

L3R 5H6 Canada
info@quanser.com
Phone: 1-905-940-3575
Fax: 1-905-940-3576

Printed in Markham, Ontario.

For more information on the solutions Quanser Inc. offers, please visit the web site at: http://www.quanser.com

This document and the software described in it are provided subject to a license agreement. Neither the software nor this document may be used or copied except as specified under the terms of that license agreement. Quanser Inc. grants the following rights: a) The right to reproduce the work, to incorporate the work into one or more collections, and to reproduce the work as incorporated in the collections, b) to create and reproduce adaptations provided reasonable steps are taken to clearly identify the changes that were made to the original work, c) to distribute and publically perform the work including as incorporated in collections, and d) to distribute and publicly perform adaptations. The above rights may be exercised in all media and formats whether now known or hereafter devised. These rights are granted subject to and limited by the following restrictions: a) You may not exercise any of the rights granted to You in above in any manner that is primarily intended for or directed toward commercial advantage or private monetary compensation, and b) You must keep intact all copyright notices for the Work and provide the name Quanser Inc. for attribution. These restrictions may not be waved without express prior written permission of Quanser Inc.